Technical University of Munich

School of Computation, Information and Technology
-- Informatics --

Bachelor's Thesis in Information Systems

AI-Driven Workflow for Inconsistency Remediation in Programming Exercises

KI-gesteuerter Workflow zur Inkonsistenzbehebung in Programmieraufgaben

Author:	Mikhail Khinevich
Supervisor:	Prof. Dr. Stephan Krusche
Advisors:	Dr. Sandro Speth, Felix T.J. Dietrich, M.Sc.
Start Date:	06.11.2025
Submission Date:	21.04.2026

Abstract

Automated quality assurance for multi-artifact programming exercises is an important component of scalable computer science education. Existing systems that use large language models (LLMs) to detect inconsistencies between exercise components demonstrate fundamental feasibility, but suffer from significant limitations in detection precision, leading to high rates of false positives. The current approach does not explicitly model the relationships between exercise components, such as problem statement, code templates, sample solutions and tests repositories.

We extend the existing system with a new approach for continuous context representation to address these limitations. This approach models semantic and structural relationships between exercise artifacts and provides the LLM-Based consistency check system with structured domain knowledge to improve the precision of detecting inconsistencies. This thesis further extends benchmark dataset towards 16 real university exercises and enables continuous evaluation of consistency check approaches, it also enables equitable comparison with the baseline. This work aims to create a more reliable and sustainable system for ensuring the quality of education.

1 Introduction

The increasing number of students studying computer science presents a serious challenge for manual grading and creation of high-quality programming exercises. Modern learning management systems (LMS) address this challenge by providing instructors with tools to manage complex exercises, consisting of multiple elements. These exercises often include components such as problem statements, code templates, sample solutions, and automated test suites that reflect real-world software development practices. Successful management of these related components is critical to ensuring effective interactive learning at scale.

Technical University of Munich developed the Artemis platform, an open-source LMS, specifically designed for this environment. It supports interactive learning and automatic assessment in programming courses, based on version

control and continuous integration [KS18]. A central feature of Artemis is its robust support for multi-artifact programming exercises, which are essential for developing practical skills in students. While Artemis automates the assessment workflow, creating high-quality complex exercises remains a challenging task for instructors.

To support this process, the Hyperion service provides artificial intelligence (AI) based assistance within the Artemis ecosystem. This intelligent tool improves the exercise creation process. It aligns with the broader goals of artificial intelligence in education [TZG22], which aims to use intelligent systems to support instructors and create personalized learning experiences. Hyperion's core service provides automated feedback to instructors on the consistency of programming exercises. It analyzes various exercise artifacts to help them identify potential issues before delivery to students.

2 Problem

Inconsistencies in multi-artifact programming exercises often arise when instructors modify one artifact, such as the problem statement, but overlook the necessary corresponding changes in others, like the code template. For example, the assignment may specify that students should implement a function named `calculateAverage`, while the provided code template contains a placeholder for a function named `computeMean` [Die+25]. These unintended inconsistencies between different artifacts of the exercise are a major problem that has significant negative consequences for the two main participants in the learning process: the students, who solve the exercises, and the instructors, who create them.

Contradictions between the assignment and the provided code template create unnecessary cognitive load for the students. This cognitive load forces them to divert intellectual resources away from the intended learning goal and direct them toward the unrelated task of aligning contradictory information. Such inconsistencies can lead to confusion, disrupt the learning process, and ultimately compromise the fairness and effectiveness of assessment [Die+25].

Instructors face the challenge of ensuring the quality and consistency of all learning materials. Detecting and resolving these issues is a mostly manual, time-consuming, and error-prone process. While Hyperion's AI-based quality assurance system aims to facilitate this workload and increase the value of automation, its performance has notable limitations, as detailed in Figure 1. The system's low precision leads to a high rate of false positives. This thesis hypothesizes that a root cause for this low precision is that the current approach treats the exercise artifacts as a single block of text, without explicitly modeling the relationships between them.

Model	TP	FP	FN	Prec.	Rec.	F1	Span F1	IoU	Time (s)	Cost ($)
OpenAI o4-mini	254	148	25	0.63	0.91	**0.75**	0.68	0.57	32.96	0.0338
xAI Grok 3 Mini	233	222	46	0.51	0.84	0.63	0.64	0.53	**14.31**	**0.0061**
Google Gemini 2.5 Flash	263	623	15	0.30	**0.95**	0.45	0.60	0.47	26.38	0.0244
Google Gemini 2.5 Flash Lite	216	288	21	0.43	0.91	0.58	0.59	0.49	16.97	0.0063

Figure 1: Overall results across the 1 049 analysed runs. F1-score measures a system's overall accuracy by calculating the harmonic mean of precision and recall. A high F1-score indicates a good balance between avoiding false positives (high precision) and not missing true issues (high recall). [Die+25].

3 Motivation

Ensuring the quality and consistency of educational materials is one of the main challenges in scalable computer science education. The scientific significance lies in the need to develop reliable automated tools capable of analyzing both natural language and formal source code. An empirical analysis of the baseline system strengthens the motivation for a new approach. This analysis, illustrated in Figure 2, reveals that the system's limitations do not significantly correlate with factors like input length, indicating that this thesis must address a more fundamental, structural problem.

Developing systems capable of bridging these structural and semantic gaps between natural language instructions and source code is a complex interdisciplinary task at the intersection of software engineering and artificial intelligence in education. A reliable solution to this problem opens up new perspectives for both

students and instructors. For students, the key benefit is a learning environment that is free from confusing and contradictory information. This allows students to engage in a seamless learning process, dedicating their full cognitive resources to mastering the necessary programming concepts, which leads to a smoother, more efficient, and fairer educational experience that focuses on skill development.

For instructors, having a reliable automated workflow is equally important. This tool provides them with an efficient and trustworthy process for ensuring the quality of their educational materials. By trusting the system with this quality control task, instructors can devote their valuable time and expertise to higher-level pedagogical tasks. This allows them to focus on designing more creative exercises and providing individualized student feedback, which is essential for the scalable delivery of high-quality computer science education.

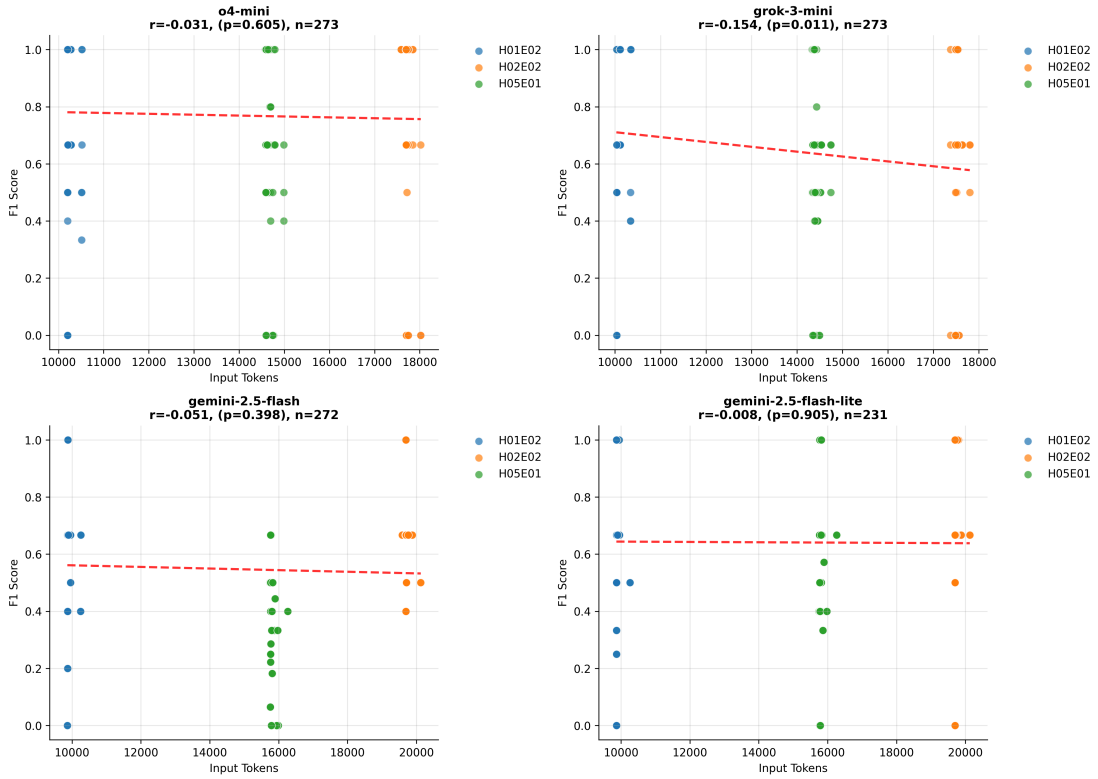

Figure 2: Relationship between prompt input token and F1-value for 4 different LLMs, revealing a weak and statistically insignificant correlation for all models.

4 Objective

This thesis aims to enhance an existing AI-driven consistency checker for programming exercises. To provide a foundation for this experimental work, the first two objectives focus on establishing a robust benchmarking pipeline and extending the evaluation dataset for more comprehensive testing. Building upon this, the third objective implements and evaluates the core technical contribution that improves detection precision. The fourth objective explores a forward-looking direction to enhance the context representation through structured artifact relationships. We define the following objectives to achieve these goals:

1. Establish an On-Demand Benchmarking Workflow.
2. Extend Consistency Check Dataset.
3. Improve Consistency Issues Detection.
4. Enhance Context Representation.

4.1 Establish an On-Demand Benchmarking Workflow

The first objective is to establish an integrated and on-demand benchmarking workflow, using PECV (Programming Exercise Consistency Verification)[1] Dataset for the Consistency Check within the Artemis platform. This approach is necessary to systematically evaluate new approaches. Instead of a continuous process, a developer will trigger this workflow on-demand through a GitHub Actions. This capability allows developers to conveniently run the benchmark against a specific version of consistency check approach to measure its performance, so that developers can contribute measurable improvements.

When a developer triggers the workflow, it will clone the necessary repositories, create an evaluation course and programming exercise for each variant in Artemis. Execute the full benchmark, and generate an evaluation report. It serves as the primary tool for evaluating new approaches this thesis develops in subsequent objectives and ensures that all results are reproducible and directly comparable to the baseline.

[1] PECV-Bench Repository

4.2 Extend Consistency Check Dataset

The second objective is to extend the existing benchmark dataset to enable a more comprehensive evaluation. The foundational dataset from [Die+25] is a crucial starting point but is limited in the scope and variety of inconsistencies it covers. This objective will focus on expanding the dataset to better reflect the range of potential issues found in real-world educational materials and support different programming languages beyond Java.

The extended dataset will draw from real exercises that different instructors at TUM created for their courses, covering multiple programming languages such as Python, C, Assembler, SQL, and Swift. Each exercise will follow the existing PECV-bench structure: a clean base exercise without inconsistencies and a set of modified variants with injected consistency issues and corresponding verified gold-standard annotations. This extended dataset will provide the necessary foundation to test the consistency checker across diverse programming environments and validate that proposed improvements generalize beyond a single programming language.

4.3 Improve Consistency Issues Detection

The third objective is to reduce the false positive rate of the consistency check system while maintaining its high recall. The baseline evaluation reveals that roughly one out of three reported issues does not correspond to a real inconsistency, which forces instructors to manually dismiss false alarms and erodes trust in the automated analysis. This thesis will pursue two complementary improvements to the detection pipeline: a language-aware context filter and a verification checker.

The language-aware context filter will extend the existing Java-only file filtering to support all programming languages in the extended dataset. The filter will apply language-specific exclusion strategies that remove build artifacts, configuration files, and binary files before the context reaches the LLM, ensuring that the model receives only relevant source code and exercise content. The verification checker will introduce an additional analysis step that runs sequentially after the

existing structural and semantic checkers. This checker will receive the issues that the first detection step produced and re-evaluate each reported issue against the exercise artifacts to identify and remove false positives before the report reaches the instructor. This approach targets the precision bottleneck directly: rather than modifying how the system detects issues, the verification checker adds a dedicated filtering stage that validates each detection.

4.4 Enhance Context Representation

The fourth objective addresses the fundamental structural problem behind the system's low detection precision. This thesis will implement and evaluate a system that uses a Knowledge Graph (KG) to formally model Artemis exercise artifacts and their interdependencies [PC24]. KG provides a machine-readable schema of the domain that captures the semantic and structural relationships between artifacts such as a problem statement, a specific method in a code template, and its corresponding test case. This approach provides the system with structured knowledge to guide its reasoning process [CJX20].

The implementation will explore a Graph Retrieval-Augmented Generation (GraphRAG) approach [ZC24]. This technique allows the LLM to query the structured KG to retrieve precise, relevant context about artifact relationships, rather than processing a single, large block of unstructured text. This method of structured information retrieval will provide the LLM with a more consistent and context-aware view of the exercise, which should reduce uncertainty, lower the false positive rate, and ultimately lead to a significant improvement in detection performance.

5 Schedule

The work for this thesis begins on November 6, 2025, and concludes on April 21, 2026. This thesis structures the project into the following nine iterations to ensure a clear focus and measurable progress throughout the five-month period:

1. **Iteration (Weeks 1-4): Benchmarking Workflow Implementation** (Objective 4.1)
   - Implement the on-demand benchmarking workflow that connects Artemis with the PECV-bench evaluation framework.
   - Automate the process of running the consistency check against the benchmark dataset through the Artemis REST API.
   - Ensure all experimental results are logged, versioned, and reproducible within the workflow.
2. **Iteration (Weeks 5-7): Dataset Expansion — Multi-Language Exercises** (Objective 4.2)
   - Collect real exercises from TUM courses across multiple programming languages (Python, C, Assembler, SQL, Swift).
   - Create exercise variants with injected consistency issues that cover the six ontology subcategories.
   - Expand the dataset to better reflect the range of potential issues found in real-world educational materials.
3. **Iteration (Weeks 8-9): Gold-Standard Annotation** (Objective 4.2)
   - Generate the corresponding gold-standard annotations for all new exercise variants.
   - Verify each annotation against the injected inconsistency to ensure evaluation reliability.
   - Finalize the extended dataset for use in subsequent evaluation tasks.
4. **Iteration (Weeks 10-11): Language-Aware Context Filter** (Objective 4.3)
   - Extend the existing Java-only context filter to support all programming languages in the extended dataset.
   - Implement language-specific exclusion strategies for build artifacts, configuration files, and binary files.
   - Evaluate the impact of the language filter on detection precision through the benchmarking workflow.

5. **Iteration (Weeks 12-14): Verification Checker** (Objective 4.3)
   - Implement the verification checker as a sequential analysis step after the parallel structural and semantic checkers.
   - Design prompts that re-evaluate each detected issue against the exercise artifacts to filter false positives.
   - Evaluate the precision improvement of the verification checker through the benchmarking workflow.
6. **Iteration (Weeks 15-17): Knowledge Graph Implementation** (Objective 4.4)
   - Design the formal schema for the Knowledge Graph to represent Artemis exercise artifacts, their dependencies and relationships.
   - Implement the logic to parse raw exercise files and populate the Knowledge Graph with the data.
7. **Iteration (Weeks 18-19): GraphRAG Pipeline Integration** (Objective 4.4)
   - Integrate the populated Knowledge Graph into the LLM analysis pipeline.
   - Engineer and test new GraphRAG prompt strategies that leverage the structured information.
8. **Iteration (Weeks 20-21): Final System Evaluation (All Objectives)**
   - Run the complete, enhanced system against the extended benchmark dataset using the on-demand benchmarking workflow.
   - Collect and analyze the final performance metrics across all pipeline iterations (precision, recall, F1, span F1, IoU).
9. **Iteration (Weeks 22-23): System Refinement and Replication Package**
   - Conduct end-to-end system testing to identify and resolve any remaining bugs.
   - Finalize the codebase and prepare the replication package with all necessary documentation.

Bibliography

[KS18] S. Krusche and A. Seitz, "ArTEMiS: An Automatic Assessment Management System for Interactive Learning," in *SIGCSE '18: Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, 2018.

[TZG22] O. Tapalova, N. Zhiyenbayeva, and D. Gura, "Artificial Intelligence in Education: AIEd for Personalised Learning Pathways," *Electronic Journal of e-Learning*, 2022.

[Die+25] F. Dietrich, Y. Zhou, T. Wasner, S. Krusche, and M. Acosta, "LLM-Based Multi-Artifact Consistency Verification for Programming Exercise Quality Assurance," 2025.

[PC24] J. Z. Pan and o. Chen, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10387715

[CJX20] X. Chen, S. Jia, and Y. Xiang, "Knowledge reasoning over knowledge graph," 2020. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0957417419306669

[ZC24] Z. Zhang and o. Chen, "GraphRAG: A Graph-Based Retrieval-Augmented Generation Method," 2024. [Online]. Available: https://arxiv.org/abs/2405.16506

Transparency in the use of AI tools

In preparing this thesis, I utilized Grammarly and Apple Intelligence for grammar and style correction across all sections, ensuring clarity and coherence in my writing. I used DeepL to enhance language quality. I used GoogleAI to generate initial drafts and expand on ideas. I have carefully checked all texts created with these tools to ensure they are correct and make sense.